4/13/2025

Mastering Oracle Performance Diagnostics

Automating ASH, AWR, and ADDM Reports with SQL and Shell Scripts

Asfaw Gedamu

Introduction

In the world of modern data systems, **database performance isn't optional. It's critical**. Oracle DBAs must quickly detect slowdowns, optimize workloads, and maintain stability under pressure.

That's where Oracle's diagnostic arsenal, **ASH, AWR, and ADDM**, comes into play. These tools deliver deep, actionable insights into performance, helping DBAs move from guesswork to precision.

This hands-on guide shows you how to **generate and automate ASH, AWR, and ADDM reports** using SQL and shell scripts. You'll learn how to time snapshots smartly, stay compliant, secure sensitive metrics, and build custom baselines that matter.

Here is your step-by-step playbook for **keeping Oracle databases fast, stable, and future-ready**.

Prerequisites

1. **Oracle Database Access** : Ensure you have the necessary privileges (SYSDBA or DBA role) to generate these reports.
2. **Environment Setup** :
    - Oracle client installed on the machine running the script.
    - $ORACLE_HOME and $PATH environment variables configured.
3. **SQL*Plus** : The Oracle command-line tool (sqlplus) must be available.
4. **Database Credentials** : Have the username, password, and database connection string ready.

Step 1: Understanding the Reports

There is a practical script set for managing Oracle ASH, AWR, and ADDM reports efficiently with SQL queries and shell scripting. It covers:

- **ASH Report** : Provides detailed information about active sessions over a specific time period.

- **AWR Report** : Generates a comprehensive performance report for a specific time range.
- **ADDM Report** : Analyzes AWR data to provide diagnostic insights and recommendations.

Step 2: SQL Queries for Generating Reports

Oracle provides PL/SQL procedures to generate these reports. Below are the key procedures:

ASH (Active Session History) Report

Purpose: Samples active session data every second and captures in-memory performance data, such as blocking sessions, top waits, top SQLs, and sessions causing the most load.

Use it when: You need fine-grained, near real-time diagnostics or you want to investigate spikes or performance anomalies within short time ranges

```
BEGIN
  DBMS_WORKLOAD_REPOSITORY.ASH_REPORT_HTML(
    l_dbid      => <DBID>,           -- Database ID
    l_inst_num  => <INSTANCE_NUMBER>, -- Instance Number
    l_btime     => TO_DATE('<BEGIN_TIME>', 'YYYY-MM-DD
HH24:MI:SS'),
    l_etime     => TO_DATE('<END_TIME>', 'YYYY-MM-DD
HH24:MI:SS'),
    l_report    => :report_clob     -- Output variable for the
report
  );
END;
/
```

AWR (Automatic Workload Repository) Report

Purpose:Provides a comprehensive performance snapshot between two time intervals (snapshots). It includes data on top SQLs, wait events, system stats, I/O, memory, and more.

Use it when: You want to analyze historical performance trends, find root causes of slowness, or compare workloads over time.

```
BEGIN
  DBMS_WORKLOAD_REPOSITORY.AWR_REPORT_HTML(
    l_dbid      => <DBID>,            -- Database ID
    l_inst_num  => <INSTANCE_NUMBER>, -- Instance Number
    l_bid       => <BEGIN_SNAP_ID>,   -- Begin Snapshot ID
    l_eid       => <END_SNAP_ID>,     -- End Snapshot ID
    l_report    => :report_clob       -- Output variable for the
report
  );
END;
/
```

ADDM (Automatic Database Diagnostic Monitor) Report

Purpose: Uses AWR data to perform an automated analysis and provides actionable tuning recommendations, e.g., reduce CPU load, fix high I/O SQLs, adjust memory settings.

Use it when: You want Oracle's expert system to advise you, especially helpful for quick diagnostics and performance improvement suggestions.

```
BEGIN
  DBMS_ADDM.ANALYZE_DB(
    task_name   => 'ADDM_TASK',
    begin_snap  => <BEGIN_SNAP_ID>,  -- Begin Snapshot ID
    end_snap    => <END_SNAP_ID>,    -- End Snapshot ID
    db_id       => <DBID>            -- Database ID
  );
END;
/

-- Retrieve ADDM Report
SELECT DBMS_ADVISOR.GET_TASK_REPORT('ADDM_TASK') AS ADDM_REPORT
FROM DUAL;
```

Step 3: Shell Script to Automate Report Generation

Below is a complete Shell script that automates the generation of ASH, AWR, and ADDM reports.

Shell Script: generate_oracle_reports.sh

```
#!/bin/bash

# Configuration
ORACLE_HOME=/path/to/oracle/home
export ORACLE_HOME
export PATH=$ORACLE_HOME/bin:$PATH
DB_USER="sys as sysdba"
DB_PASSWORD="your_password"
DB_CONNECTION="your_database_connection_string"

# Input Parameters
DBID=$1
INSTANCE_NUM=$2
BEGIN_TIME=$3
END_TIME=$4
BEGIN_SNAP_ID=$5
END_SNAP_ID=$6

# Directory for Reports
REPORT_DIR="/path/to/reports"
mkdir -p $REPORT_DIR

# Generate ASH Report
echo "Generating ASH Report..."
sqlplus -s ${DB_USER}/${DB_PASSWORD}@${DB_CONNECTION} <<EOF >
${REPORT_DIR}/ash_report.html
SET SERVEROUTPUT ON
SET LONG 1000000
SET PAGESIZE 0
SET LINESIZE 1000
SPOOL ${REPORT_DIR}/ash_report.html
DECLARE
  report_clob CLOB;
BEGIN
  DBMS_WORKLOAD_REPOSITORY.ASH_REPORT_HTML(
    l_dbid      => $DBID,
```

```
    l_inst_num  => $INSTANCE_NUM,
    l_btime     => TO_DATE('$BEGIN_TIME', 'YYYY-MM-DD
HH24:MI:SS'),
    l_etime     => TO_DATE('$END_TIME', 'YYYY-MM-DD
HH24:MI:SS'),
    l_report    => report_clob
  );
  DBMS_OUTPUT.PUT_LINE(report_clob);
END;
/
SPOOL OFF
EXIT;
EOF

# Generate AWR Report
echo "Generating AWR Report..."
sqlplus -s ${DB_USER}/${DB_PASSWORD}@${DB_CONNECTION} <<EOF >
${REPORT_DIR}/awr_report.html
SET SERVEROUTPUT ON
SET LONG 1000000
SET PAGESIZE 0
SET LINESIZE 1000
SPOOL ${REPORT_DIR}/awr_report.html
DECLARE
  report_clob CLOB;
BEGIN
  DBMS_WORKLOAD_REPOSITORY.AWR_REPORT_HTML(
    l_dbid      => $DBID,
    l_inst_num  => $INSTANCE_NUM,
    l_bid       => $BEGIN_SNAP_ID,
    l_eid       => $END_SNAP_ID,
    l_report    => report_clob
  );
  DBMS_OUTPUT.PUT_LINE(report_clob);
END;
/
SPOOL OFF
EXIT;
EOF

# Generate ADDM Report
```

```
echo "Generating ADDM Report..."
sqlplus -s ${DB_USER}/${DB_PASSWORD}@${DB_CONNECTION} <<EOF >
${REPORT_DIR}/addm_report.txt
SET SERVEROUTPUT ON
SET LONG 1000000
SET PAGESIZE 0
SET LINESIZE 1000
SPOOL ${REPORT_DIR}/addm_report.txt
BEGIN
  DBMS_ADDM.ANALYZE_DB(
    task_name   => 'ADDM_TASK',
    begin_snap  => $BEGIN_SNAP_ID,
    end_snap    => $END_SNAP_ID,
    db_id       => $DBID
  );
END;
/
SELECT DBMS_ADVISOR.GET_TASK_REPORT('ADDM_TASK') AS ADDM_REPORT
FROM DUAL;
SPOOL OFF
EXIT;
EOF

echo "Reports generated successfully in $REPORT_DIR"
```

Step 4: Running the Script

1. Save the script as generate_oracle_reports.sh.
2. Make it executable:

   ```
   chmod +x generate_oracle_reports.sh
   ```

3. Run the script with required parameters:
   Example:

   ```
   ./generate_oracle_reports.sh <DBID> <INSTANCE_NUM>
   "<BEGIN_TIME>" "<END_TIME>" <BEGIN_SNAP_ID> <END_SNAP_ID>
   ```

Step 5: Verifying the Reports

- The generated reports (ash_report.html, awr_report.html, addm_report.txt) will be saved in the specified REPORT_DIR.
- Open the HTML files in a browser for better readability.

Bonus Tips

- Automate the script with cron for daily performance tracking.
- Use sqlplus -S to silence output and SPOOL to capture reports.
- You can dynamically get the latest snapshot IDs:

Example:

Crontab: Schedule the Job

```
# Run this weekly or daily based on your strategy:

crontab -e

0 8 * * * /home/oracle/scripts/generate_and_email_awr.sh >>
/home/oracle/logs/awr_cron.log 2>&1

# Ensure the script is owned by Oracle and not writable by
others:
chmod 700 generate_and_email_awr.sh
chown oracle:oinstall generate_and_email_awr.sh
```

Notes

1. **Snapshot IDs** : Use the following query to find snapshot IDs:

   ```
   SELECT SNAP_ID, BEGIN_INTERVAL_TIME, END_INTERVAL_TIME
   FROM DBA_HIST_SNAPSHOT
   ORDER BY SNAP_ID DESC;
   ```

2. **Permissions** : Ensure the user has sufficient privileges to execute DBMS_WORKLOAD_REPOSITORY and DBMS_ADDM procedures.

3. **Error Handling** : Add error handling in the script to capture and log any issues during execution.

The provided Shell script and SQL queries allow you to generate **ASH** , **AWR** , and **ADDM** reports programmatically. These tools are essential for diagnosing and optimizing Oracle database performance.

Moreover DBAs Consider the following:

1. **Snapshot Timing:**
   Choose meaningful intervals — not just arbitrary hours. Use workload events (peak traffic, backup windows) to time snapshots.
2. **Licensing Compliance:**
   AWR/ASH/ADDM are part of the **Diagnostic Pack** — ensure your environment is licensed for it.
3. **Custom Baselines:**
   Establish performance baselines for comparison, especially before/after migrations, patching, or architectural changes.
4. **Retention & Storage:**
   Manage AWR snapshot retention policies to balance history depth with storage usage.
5. **Security & Access:**
   Limit report access — these contain sensitive internal metrics.
6. **Integrate with Monitoring Tools:**
   Connect insights from AWR/ASH/ADDM with tools like Oracle Enterprise Manager, Grafana, or Prometheus for holistic views.

1. Snapshot Timing

Snapshots should be timed based on workload events (e.g., peak traffic or backup windows). Use Oracle's DBMS_WORKLOAD_REPOSITORY package to manage snapshots.

SQL Query: Modify Snapshot Interval

```
BEGIN
  DBMS_WORKLOAD_REPOSITORY.MODIFY_SNAPSHOT_SETTINGS(
```

```
    retention => 43200, -- Retention in minutes (e.g., 30 days)
    interval  => 60     -- Interval in minutes (e.g., every
hour)
  );
END;

/
```

Shell Script: Automate Snapshot Timing Based on Workload Events

This script schedules snapshots during peak traffic hours (e.g., 9 AM–5 PM) and backup windows (e.g., midnight).

```
#!/bin/bash

# Oracle environment setup
export ORACLE_SID=your_oracle_sid
export ORACLE_HOME=/path/to/oracle/home
export PATH=$ORACLE_HOME/bin:$PATH

# Function to take a snapshot
take_snapshot() {
  sqlplus -s / as sysdba <<EOF
  BEGIN
    DBMS_WORKLOAD_REPOSITORY.CREATE_SNAPSHOT();
  END;
  /
  EXIT;
EOF
}

# Schedule snapshots during peak traffic hours (9 AM - 5 PM)
for hour in {9..17}; do
  echo "Taking snapshot at $hour:00"
  take_snapshot
done

# Schedule snapshot during backup window (midnight)
echo "Taking snapshot at midnight"
take_snapshot
```

2. Licensing Compliance

Ensure your environment is licensed for AWR/ASH/ADDM by querying licensing information.

SQL Query: Check Diagnostic Pack License

```
SELECT *
FROM dba_feature_usage_statistics
WHERE name IN ('Automatic Workload Repository', 'Active Session
History', 'Automatic Database Diagnostic Monitor');
```

Shell Script: Alert for Licensing Issues

This script checks if the required features are licensed and sends an alert if they are not.

```
#!/bin/bash

# Oracle environment setup
export ORACLE_SID=your_oracle_sid
export ORACLE_HOME=/path/to/oracle/home
export PATH=$ORACLE_HOME/bin:$PATH

# Query licensing status
LICENSE_STATUS=$(sqlplus -s / as sysdba <<EOF
SET HEADING OFF
SET FEEDBACK OFF
SET PAGESIZE 0
SELECT DECODE(COUNT(*), 0, 'NOT LICENSED', 'LICENSED')
FROM dba_feature_usage_statistics
WHERE name IN ('Automatic Workload Repository', 'Active Session
History', 'Automatic Database Diagnostic Monitor');
EXIT;
EOF
)

# Check and alert
if [ "$LICENSE_STATUS" == "NOT LICENSED" ]; then
  echo "ALERT: Your environment is NOT LICENSED for
```

```
AWR/ASH/ADDM."
else
  echo "Your environment is properly licensed for AWR/ASH/ADDM."
fi
```

3. Custom Baselines

Establish performance baselines for comparison before/after migrations, patching, or architectural changes.

SQL Query: Create a Baseline

```
BEGIN
  DBMS_WORKLOAD_REPOSITORY.CREATE_BASELINE(
    start_snap_id => 100, -- Start snapshot ID
    end_snap_id   => 200, -- End snapshot ID
    baseline_name => 'Pre_Migration_Baseline'
  );
END;
/
```

Shell Script: Automate Baseline Creation

This script creates a baseline for a specific time range.

```
#!/bin/bash

# Oracle environment setup
export ORACLE_SID=your_oracle_sid
export ORACLE_HOME=/path/to/oracle/home
export PATH=$ORACLE_HOME/bin:$PATH

# Input parameters
START_SNAP_ID=$1
END_SNAP_ID=$2
BASELINE_NAME=$3
```

```
# Create baseline
sqlplus -s / as sysdba <<EOF
BEGIN
  DBMS_WORKLOAD_REPOSITORY.CREATE_BASELINE(
    start_snap_id => $START_SNAP_ID,
    end_snap_id   => $END_SNAP_ID,
    baseline_name => '$BASELINE_NAME'
  );
END;
/
EXIT;
EOF
```

4. Retention & Storage

Manage AWR snapshot retention policies to balance history depth with storage usage.

SQL Query: Adjust Retention Policy

```
BEGIN
  DBMS_WORKLOAD_REPOSITORY.MODIFY_SNAPSHOT_SETTINGS(
    retention => 8640, -- Retention in minutes (e.g., 6 days)
    interval  => 30    -- Interval in minutes (e.g., every 30
minutes)
  );
END;
/
```

Shell Script: Automate Retention Management

This script adjusts retention settings based on storage thresholds.

```
#!/bin/bash

# Oracle environment setup
export ORACLE_SID=your_oracle_sid
export ORACLE_HOME=/path/to/oracle/home
export PATH=$ORACLE_HOME/bin:$PATH

# Set retention policy
RETENTION_MINUTES=8640 # 6 days
```

```
INTERVAL_MINUTES=30    # Every 30 minutes

sqlplus -s / as sysdba <<EOF
BEGIN
  DBMS_WORKLOAD_REPOSITORY.MODIFY_SNAPSHOT_SETTINGS(
    retention => $RETENTION_MINUTES,
    interval  => $INTERVAL_MINUTES
  );
END;
/
EXIT;
EOF
```

5. Security & Access

Limit access to sensitive metrics in AWR/ASH/ADDM reports.

SQL Query: Restrict Access

```
REVOKE SELECT ON dba_hist_snapshot FROM PUBLIC;
GRANT SELECT ON dba_hist_snapshot TO restricted_user;
```

Shell Script: Audit Access Permissions

This script audits who has access to AWR/ASH/ADDM data.

```
#!/bin/bash

# Oracle environment setup
export ORACLE_SID=your_oracle_sid
export ORACLE_HOME=/path/to/oracle/home
export PATH=$ORACLE_HOME/bin:$PATH

# Query access permissions
ACCESS_PERMISSIONS=$(sqlplus -s / as sysdba <<EOF
SET LINESIZE 200
SET PAGESIZE 100
SELECT grantee, privilege
FROM dba_tab_privs
```

```
WHERE table_name = 'DBA_HIST_SNAPSHOT';
EXIT;
EOF
)

# Display results
echo "Access Permissions for AWR/ASH/ADDM:"
echo "$ACCESS_PERMISSIONS"
```

6. Integrate with Monitoring Tools

Connect insights from AWR/ASH/ADDM with tools like Oracle Enterprise Manager, Grafana, or Prometheus.

SQL Query: Export Data for Integration

Export AWR/ASH data to CSV for integration with monitoring tools.

```
SPOOL awr_data.csv
SELECT * FROM dba_hist_snapshot;
SPOOL OFF
```

Shell Script: Automate Data Export

This script exports AWR data to a CSV file for ingestion into monitoring tools.

```
#!/bin/bash

# Oracle environment setup
export ORACLE_SID=your_oracle_sid
export ORACLE_HOME=/path/to/oracle/home
export PATH=$ORACLE_HOME/bin:$PATH

# Export AWR data
sqlplus -s / as sysdba <<EOF
SET HEADING OFF
SET FEEDBACK OFF
SET PAGESIZE 0
SET COLSEP ','
SPOOL awr_data.csv
SELECT * FROM dba_hist_snapshot;
```

```
SPOOL OFF
EXIT;
EOF

# Move exported file to monitoring tool directory
mv awr_data.csv /path/to/monitoring/tool/input/
```

Conclusion

The above SQL queries and shell scripts provide a comprehensive solution for managing **AWR/ASH/ADDM** snapshots, ensuring licensing compliance, creating custom baselines, managing retention policies, securing access, and integrating with monitoring tools. These scripts can be customized further based on specific requirements.

If you like this content, you can download it along with other documents from:

https://t.me/paragonacademy